

```
$ sudo apt-get install libxine1-
ffmpeg libxine-dev libxine1-bin
$ sudo apt-get install libunicap2
libunicap2-dev
$ sudo apt-get install libdcl394-22-
dev libdcl394-22 libdcl394-utils
$ sudo apt-get install swig
$ sudo apt-get install libv4l-0
libv4l-dev
$ sudo apt-get install python-numpy
$ sudo apt-get install build-essential
libgtk2.0-dev libjpeg62-dev libtiff4-
dev libjasper-dev libopenexr-dev
cmake python-dev python-numpy libtbb-
dev libeigen2-dev yasm libfaac-dev
libopencore-amrnb-dev libopencore-
amrwb-dev libtheora-dev libvorbis-dev
libxvid-dev
```

2. Install Python development headers with 'sudo apt-get install python-dev.'

3. Download the source code from <http://sourceforge.net/projects/opencvlibrary/files/opencv-unix/2.3/>. It is recommended that you move the downloaded OpenCV package to your /home/<user> directory, extract the archive, as follows:

```
$ tar -xvf OpenCV-2.3.1a.tar.bz2
cd OpenCV-2.3.1/
```

4. Make a new directory called 'build' and change directory into it (that is, mkdir build; cd build).

5. Run Cmake as given below:

```
$ cmake -D WITH_TBB=ON -D BUILD_NEW_
PYTHON_SUPPORT=ON -D WITH_V4L=OFF-D
INSTALL_C_EXAMPLES=ON -D INSTALL_
PYTHON_EXAMPLES=ON -D BUILD_
EXAMPLES=ON ..
```

6. Follow this with 'make' and 'sudo make install' (to install the library).

7. Next, configure the system to use the new OpenCV shared libraries. Edit a configuration file with `sudo gedit /etc/ld.so.conf.d/opencv.conf`. Add the following line at the end of the file (it may be an empty file, which is okay) and then save it:

```
/usr/local/lib
```

Close the file and run `sudo ldconfig`.

8. Open your system `bashrc` file (for me, that is `sudo gedit /etc/bash.bashrc`) and add the following code at the end



Fig. 2: Reading and displaying the image



Fig. 3: Gray scale image

of the file:

```
PKG_CONFIG_PATH=$PKG_CONFIG_PATH:
/usr/local/lib/pkgconfig
export PKG_CONFIG_PATH
```

9. Save and close the file, then log out and log in again, or reboot the system.

Examples of image processing

Now let us see how to use the OpenCV library via the Python interface to perform image processing-based tasks.

Here is an explanation of OpenCV syntax and how to use it, along with a few examples. We need not go into the basics of Python code and syntax since that is extensively documented; see <http://www.python.org/doc/> for more.

Reading and displaying an image. Our first example named 'test.py' is written in Python (refer EYF Note about the source code of this article).

Open the terminal, navigate to the directory that contains the test.py file and run it with 'python test.py' command. Here, tnpng is the name of the image file stored under home/jayneil/ folder. You can change the name of the folder if you wish, but you will have to change it in the source code too. The output is as shown in Fig. 2.

The functions in test.py code can be explained as given below.

The 'Import' function imports the OpenCV module 'cv' so that you can use it, then call the 'LoadImage' function (passing the full path to the file,

which is needed) to load the image into the variable src.

The 'Named-Window' function creates a window to display the loaded image. The first parameter is the name for the window, and the second sets the sizing mode to automatically resize the window to the size of the image.

'ShowImage' is used to display the

image in the window, passing it the name of the window and the image variable.

'WaitKey' is used so that you can see the image for a sizeable period of time (similar to a delay function).

Finally, 'DestroyWindow' destroys the window and frees the resources it used.

Converting an RGB image to gray scale format. Most digital images use the RGB colour model, in which every pixel has three components—red, green and blue. By varying the amount of each component, a pixel gets a unique colour.

Grayscale is another popular colour model, with only one component that varies from 0-255; 0 represents black, and 255 white (for an 8-bit image). The rgb2gray.py code converts an image from RGB to gray scale, which is essential for image detection.

Running this code will give output as shown in Fig. 3. In the eighth line, we have created our own image, similar to the source image (src). The 'GetSize' returns the properties of the source image—so our new image has the same dimensions as the source. 'CreateImage' takes three parameters: the image dimensions, the colour depth of the image (8-bit or 16-bit) and the number of channels (which, for gray scale, is one). We used the 'CvtColor' function to convert the image to gray scale. This function has three parameters. The first one is the source